

ContentCachingRequestWrapper & ContentCachingResponseWrapper — Detailed Notes

1. Background: Why These Classes Exist

The Core Problem: In a Servlet-based application, the **request body** (`InputStream`) and **response body** (`OutputStream`) are designed to be **read only once**.

- `HttpServletRequest.getInputStream()` / `getReader()` — once read, the stream is **consumed**. If a Filter reads it (e.g., for logging), the Controller downstream will get an **empty stream** and fail to read the actual request body.
- `HttpServletResponse.getOutputStream()` / `getWriter()` — data written to it goes **directly to the client**. There's no built-in way to "peek" at what was written after the Controller has already sent the response.

This is a major problem when you want to build a **logging filter** that needs to print the full request/response body — because reading these streams for logging purposes would normally **break** the actual request/response flow for the rest of the application.

The Solution: Spring provides two wrapper classes (in package `org.springframework.web.util`) that solve this exact problem by **caching** the body content in memory as it's read/written, **without disturbing** the original stream behavior for downstream/upstream components.

Class	Wraps	Purpose
<code>ContentCachingRequestWrapper</code>	<code>HttpServletRequest</code>	Caches the request body as it is read, so it can be accessed multiple times (e.g., for logging)
<code>ContentCachingResponseWrapper</code>	<code>HttpServletResponse</code>	Caches the response body as it is written, so it can be inspected before actually sending it to the client

Both extend standard Servlet wrapper classes:

```
HttpServletRequestWrapper → ContentCachingRequestWrapper
HttpServletResponseWrapper → ContentCachingResponseWrapper
```

(These base wrapper classes are part of the standard **Decorator Pattern** support built into the Servlet API — designed specifically so you can wrap and augment request/response behavior.)

2. ContentCachingRequestWrapper — Detailed

What It Does

- Wraps the original `HttpServletRequest` .
- As the request `InputStream` is read (by Jackson during deserialization, or by any code reading the body), a **copy of the bytes is stored internally** in a byte array buffer — **in addition to** passing the data along normally.
- After the Controller has processed the request, you can call `getContentAsByteArray()` to retrieve the **full cached body** — even though it has already been "consumed" by the Controller.

Key Methods

Method	Purpose
<code>getContentAsByteArray()</code>	Returns the cached request body as a <code>byte[]</code> — only contains data that has actually been read so far
<code>getContentAsString()</code> ... <i>(not directly available — you build a String from the byte array using the correct charset)</i>	—
<code>getCharacterEncoding()</code>	Returns the request's character encoding (needed to convert bytes to String correctly)

Important Caveat: The cache only contains bytes that have been **read** by something (e.g., Jackson's deserializer reading `@RequestBody`). If nothing reads the input stream, `getContentAsByteArray()` returns an **empty array** — this is a very common gotcha.

Constructor Options

```
new ContentCachingRequestWrapper(request); // no size limit
new ContentCachingRequestWrapper(request, contentCacheLimit); // limits cache size in bytes
```

- The size-limited constructor is useful for large file uploads — you don't want to cache gigabytes of data into memory just for logging.

Example Usage

```
public class RequestLoggingFilter extends OncePerRequestFilter {

    @Override
    protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response,
        FilterChain filterChain) throws ServletException, IOException {

        ContentCachingRequestWrapper wrappedRequest = new ContentCachingRequestWrapper(request);

        // Pass the WRAPPED request down the chain (Controller reads from this wrapper)
        filterChain.doFilter(wrappedRequest, response);

        // AFTER the Controller has read the body, the wrapper now has it cached
        byte[] content = wrappedRequest.getContentAsByteArray();
        String requestBody = new String(content, wrappedRequest.getCharacterEncoding());

        System.out.println("Request Body: " + requestBody);
    }
}
```

Critical detail: You must call `filterChain.doFilter(wrappedRequest, response)` — passing the **wrapped** request, not the original — so that the Controller actually reads through the wrapper (which is what populates the cache). If you pass the original `request` object instead, nothing gets cached.

Also critical: You must read `getContentAsByteArray()` **AFTER** `chain.doFilter()` returns — not before — because the body is only cached as it's being read by the Controller/Jackson during request processing.

3. ContentCachingResponseWrapper — Detailed

What It Does

- Wraps the original `HttpServletResponse`.
- As the Controller writes data to the response (via `getWriter()` or `getOutputStream()`), the wrapper **buffers** this data internally **instead of** sending it directly to the client.
- After the Controller has finished writing, you can inspect the cached response body via `getContentAsByteArray()`.
- **Crucially:** You must manually call `copyBodyToResponse()` at the end — otherwise, the **actual client never receives any response body** (since it was being buffered instead of sent directly)!

Key Methods

Method	Purpose
<code>getContentAsByteArray()</code>	Returns the cached response body written so far, as <code>byte[]</code>
<code>copyBodyToResponse()</code>	Flushes the cached buffer to the real underlying response — MUST be called, or the client gets an empty response body

Example Usage

```

public class ResponseLoggingFilter extends OncePerRequestFilter {

    @Override
    protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response,
        FilterChain filterChain) throws ServletException, IOException {

        ContentCachingResponseWrapper wrappedResponse = new ContentCachingResponseWrapper(response);

        // Pass the WRAPPED response down the chain (Controller writes to this wrapper)
        filterChain.doFilter(request, wrappedResponse);

        // AFTER Controller has written the response, inspect the cached body
        byte[] content = wrappedResponse.getContentAsByteArray();
        String responseBody = new String(content, wrappedResponse.getCharacterEncoding());

        System.out.println("Response Body: " + responseBody);

        // CRITICAL STEP – actually send the buffered content to the real client
        wrappedResponse.copyBodyToResponse();
    }
}

```

Why `copyBodyToResponse()` is critical: Since the wrapper intercepted all writes and stored them in its internal buffer **instead of** writing directly to the real `HttpServletResponse`'s stream, the real response stream is **empty** until you explicitly copy the buffered bytes over. Forgetting this call is one of the most common bugs when using this class – it results in the client receiving a technically successful (200 OK) but **completely empty** response body.

4. Combined Usage – Full Request + Response Logging Filter

This is the most common real-world pattern – wrapping **both** request and response together in a single filter:

```

@Component
public class RequestResponseLoggingFilter extends OncePerRequestFilter {

    @Override
    protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response,
        FilterChain filterChain) throws ServletException, IOException {

        ContentCachingRequestWrapper wrappedRequest = new ContentCachingRequestWrapper(request);
        ContentCachingResponseWrapper wrappedResponse = new ContentCachingResponseWrapper(response);

        long startTime = System.currentTimeMillis();

        // Pass wrapped objects down the chain
        filterChain.doFilter(wrappedRequest, wrappedResponse);

        long duration = System.currentTimeMillis() - startTime;

        // Log Request
        String requestBody = new String(
            wrappedRequest.getContentAsByteArray(), wrappedRequest.getCharacterEncoding());
        System.out.println("Request [" + request.getMethod() + " " + request.getRequestURI() + "]: " + requestBody);

        // Log Response
        String responseBody = new String(
            wrappedResponse.getContentAsByteArray(), wrappedResponse.getCharacterEncoding());
        System.out.println("Response [" + wrappedResponse.getStatus() + ", " + duration + "ms]: " + responseBody);

        // MUST copy the response body back – otherwise client gets nothing
        wrappedResponse.copyBodyToResponse();
    }
}

```

Flow Diagram:

```

Client Request
↓
Filter wraps Request → ContentCachingRequestWrapper
Filter wraps Response → ContentCachingResponseWrapper
↓
chain.doFilter(wrappedRequest, wrappedResponse)
↓
DispatcherServlet → Controller
|
|— Controller reads @RequestBody
|   → bytes get cached inside ContentCachingRequestWrapper as they're read
|
|— Controller writes response (JSON, etc.)
|   → bytes get buffered inside ContentCachingResponseWrapper (NOT sent yet)
↓
Control returns to Filter (after chain.doFilter() call)
↓
Filter reads wrappedRequest.getContentAsByteArray() → logs request body
Filter reads wrappedResponse.getContentAsByteArray() → logs response body
↓
Filter calls wrappedResponse.copyBodyToResponse()
→ NOW the buffered data is actually flushed to the real client connection
↓
Client receives the Response

```

5. Use Cases

Use Case	How These Wrappers Help
Request/Response Logging	Log full request and response bodies for debugging, auditing, or monitoring — without breaking the actual data flow to Controllers/clients
API Auditing	Persist request/response payloads to a database or log aggregation system (e.g., ELK stack, Splunk) for compliance/audit trails
Debugging in Production	Capture exact payloads that caused errors, without needing to reproduce the issue manually
Performance Monitoring	Combine with timestamps to log request size, response size, and processing time together
Security/Fraud Detection	Inspect payloads for suspicious patterns (SQL injection strings, XSS payloads) as part of a security filter
Masking Sensitive Data in Logs	Read cached body, redact sensitive fields (passwords, card numbers) before logging, without altering what's actually sent to the Controller/client
Correlating Requests in Microservices	Log full payloads alongside a correlation/trace ID for distributed tracing across services

6. Common Pitfalls / Gotchas

Pitfall	Explanation
Passing original request / response instead of wrapped ones to <code>chain.doFilter()</code>	If you don't pass the wrapped objects downstream, nothing gets cached — the wrapper only "sees" data that flows through it
Reading <code>getContentAsByteArray()</code> before <code>chain.doFilter()</code> completes	The cache is populated progressively as data is read/written — reading too early returns incomplete/empty data
Forgetting <code>copyBodyToResponse()</code>	The client receives an empty response body , even though your Controller "returned" data correctly — because it was buffered, not sent

Catching very large request/response bodies (e.g., file uploads/downloads)	Explanation Can cause high memory usage — use the size-limited constructor for <code>ContentCachingRequestWrapper</code> , and generally avoid wrapping responses for large file downloads
Assuming <code>getContentAsByteArray()</code> on the request always has data	If nothing actually reads the request body (e.g., a GET request with no body, or a Controller method that doesn't use <code>@RequestBody</code>), the cache remains empty — this is expected behavior, not a bug
Character encoding mismatches	Always use <code>wrapper.getCharacterEncoding()</code> when converting the byte array to a <code>String</code> , rather than assuming UTF-8, to avoid corrupted logs for non-UTF-8 clients

7. Where to Register This Filter

Like any filter, register it via `@Component + @Order`, or `FilterRegistrationBean`:

```
@Configuration
public class FilterConfig {

    @Bean
    public FilterRegistrationBean<RequestResponseLoggingFilter> loggingFilter() {
        FilterRegistrationBean<RequestResponseLoggingFilter> registrationBean = new FilterRegistrationBean<>();
        registrationBean.setFilter(new RequestResponseLoggingFilter());
        registrationBean.addUrlPatterns("/api/*");
        registrationBean.setOrder(1);
        return registrationBean;
    }
}
```

- It should typically be placed **early** in the filter chain (low order value) so it wraps the request/response **before** any other filters or the Controller touch them — ensuring the full body is captured.

8. Class Hierarchy Recap

```
ServletRequestWrapper      ServletResponseWrapper
      ↑                      ↑
HttpServletRequestWrapper  HttpServletResponseWrapper
      ↑                      ↑
ContentCachingRequestWrapper ContentCachingResponseWrapper
```

Both are part of `org.springframework.web.util` package — provided by **Spring**, not raw Servlet API (though they build on top of the standard Servlet wrapper classes).

9. Quick Summary (for revision)

- **Problem they solve:** Request/response streams can normally only be read/written **once** — making logging of bodies impossible without breaking the actual flow.
- **ContentCachingRequestWrapper** — wraps `HttpServletRequest`; caches request body bytes **as they are read** by downstream code (like Jackson). Access via `getContentAsByteArray()` **after** `chain.doFilter()`.
- **ContentCachingResponseWrapper** — wraps `HttpServletResponse`; buffers response body bytes **as they are written** by the Controller. Access via `getContentAsByteArray()`, but **must call** `copyBodyToResponse()` afterward, or the client gets an empty response.
- **Must pass the wrapped objects** into `chain.doFilter(wrappedRequest, wrappedResponse)` — not the originals — for caching to actually work.
- **Primary use case:** Building logging/auditing filters that capture full request and response payloads without disrupting the app's normal request/response flow.
- **Biggest gotcha:** Forgetting `copyBodyToResponse()` silently breaks your API responses (empty body sent to client) — this is the #1 mistake developers make with these classes.
- Best practice: use the size-limited constructor for requests to avoid excessive memory usage on large payloads (e.g., file uploads).